

Sistemas Operativos

Práctico 3: Procesos threads y concurrencia

Ejercicio 1: Ejecutar el comando `ps aux` para ver el estado de los procesos ejecutándose en el sistema. Hacer `man ps` para ver su utilización y comprender su salida.

1. Identificar los diferentes estados de los procesos.
2. Ordenar la salida por mayor uso de CPU y luego por mayor uso de memoria.
3. Listar los procesos lanzados por su usuario en el sistema.

Ejercicio 2: Descargar el script de shell `runseconds.sh` dado, el cual ejecuta un ciclo de espera de 120 segundos usando la CPU (busy waiting).

1. Lanzarlo como proceso de fondo.
2. Verificar su estado RUNNING (con `ps ax <PID>`).
3. Pasarlo a estado STOPPED enviándole una señal con el comando `kill`. Verificar con el comando `ps` que cambió su estado (T).
4. Hacer que continúe su ejecución (determinar la señal a enviarle correspondiente). Volver a verificar que su estado es RUNNING.
5. Esperar a su terminación o terminarlo con `kill`.

Ejercicio 3: Usar el comando `ps tree` para visualizar el árbol de procesos del sistema. Determinar cuál es el primer proceso lanzado en su sistema y ver su *pid*.

Ejercicio 4: Realizar un experimento de planificación de cpu en GNU-Linux con un proceso de uso intensivo de cpu. Crear un script de shell o programa que cicle infinitamente.

1. Ejecutarlo normalmente y en otra terminal ver el uso de cpu con el comando `top`.
2. Correr otra instancia del programa con menor prioridad (ej: `nice -n 15 bash <your-program>`). Observar los procesos nuevamente con `top` y la cantidad de cambios de contexto realizados por el scheduler con `cat /proc/<PID>/sched | grep nr_switches`.
¿El segundo proceso debería ejecutar más o menos cambios de contexto por unidad de tiempo? Tener en cuenta que el kernel Linux establecer un *quantum* (o *time slice*) proporcional a su prioridad.
3. Cambiar la prioridad de una de las instancias en ejecución (ver el comando `renice`) y observar nuevamente su comportamiento viendo las estadísticas de *scheduling*.
4. Hacer que un proceso ejecute con mayor prioridad (menor valor de `nice`) que la de un proceso normal (ejemplo: con un valor de `niceness` de -10). En GNU-Linux las prioridades van de -19 (mayor prioridad) a 20 (menor prioridad).
Describe por qué cree que el sistema pide correr el comando con un usuario con mayor privilegio (usando `sudo` el sistema lo permite).

Nota: Es posible monitorear los cambios de un archivo (por ejemplo `/proc/<PID>/sched`) usando el comando `watch -n<seconds> <file>`.

Ejercicio 5: Dado el programa *counter.c*, compilarlo en el ejecutable *counter*, crear el archivo *counter.dat* el cual debe contener el valor inicial 0 (como texto) y realizar los siguientes experimentos:

1. Ejecutar el comando `./counter` ; `./counter` y analizar la salida. Debería quedar en 2000.
2. Reinicializar el valor en el archivo de datos y disparar dos o más procesos concurrentes (`./counter & ./counter`).
3. Analizar y dar una explicación del resultado obtenido en *counter.dat*. Dar
4. Solucionar el problema de concurrencia usando la función `flock()` de UNIX (ver las páginas del man). Encierre la mínima *región crítica* posible.
5. ¿Es fd una variable compartida por diferentes instancias de ejecución del programa? Justificar la respuesta.

Ejercicio 6: Ejecutar el programa *pthread_example.c*.

1. ¿Funciona correctamente? Si no, explique la causa dando algunas trazas de ejecución mostrando el problema.
2. Modificarlo para que funcione correctamente. Ayuda: Usar pthreads mutexes. Ver https://www.eecg.toronto.edu/~amza/ece1747h/homeworks/pthreads_man.html.
3. Mostrar la dirección de la variable local `tid` en la función `increment` y mostrar su dirección. Determinar la distancia entre las direcciones. ¿Considera que están en la misma pila? Justificar.
4. Mostrar con un experimento en el programa que todos los threads ejecutan en el mismo espacio de memoria del proceso.

Ejercicio 7: Java soporta programas multithreading (mediante la implementación de la interfaz `Runnable` o *heredando* de la clase `Thread`). También ofrece mecanismos de sincronización como la palabra clave `synchronize` que se aplica a clases, métodos y bloques de código. También ofrece los métodos para sincronizar threads: `wait()`, `notify()` and `notifyAll()`.

1. Analizar los mecanismos de manejo (creación y control) de threads.
2. Analizar los mecanismos de sincronización provistos. En particular clasificarlos en base los tipos de mecanismos analizados.
3. Hacer un programa Java equivalente al dado en el ejercicio 6 que no tenga *race conditions*.

Ejercicio 8: Describa las operaciones realizadas por un kernel cuando un proceso es interrumpido por el *timer* y ya usó su *quantum* hasta retornar de la interrupción. En particular, describir en qué estado el kernel deja el proceso.

Ejercicio 9: Idem al ejercicio anterior pero cuando un proceso ejecuta el `syscall` `read` (lee de un archivo en disco). Recuerde que ésta operación puede requerir mucho tiempo. Además describa cómo el proceso continuará su ejecución.

Ejercicio 10: Mostrar de manera semi-formal que la solución de Peterson (con dos procesos) garantiza exclusión mutua y progreso.

Ejercicio 11: Implementar el problema del productor-consumidor usando la API de semáforos POSIX (`sem_open`, `sem_init`, `sem_wait`, `sem_post`, ...). Ver https://man7.org/linux/man-pages/man7/sem_overview.7.html.

1. Usando POSIX threads.
2. Usando procesos y named semaphores. El buffer compartido puede ser un archivo o una shared memory area (ver https://man7.org/linux/man-pages/man7/shm_overview.7.html)

Ejercicio 12: Dado el siguiente fragmento de código de un SO. La función `interrupt_handler()` se dispara ante la ocurrencia en una interrupción de un dispositivo.

```
lock l = 0;
resource r;
```

```
void interrupt_handler()
{
    acquire(&l);
    use_resource(r);
    release(&l);
}
...
void f()
{
    // do something...
    acquire(&l);
    use_resource(&l);
    release(&l);
}

void acquire(lock *l)
{
    while (atomic_swap(l,1)==1)
        ;
}

void release(lock *l)
{
    *l = 0;
}
```

- A. Dar una traza que muestre que puede ocurrir un *deadlock*.
- B. ¿Puede considerarse un *livelock*?
- C. Proponga una corrección en la implementación de `acquire()` y `release()` para evitarlo.

Ejercicio 13: Dar el grafo que modela el uso y requerimientos de recursos e indicar si hay *deadlock*.

1. P1 adquiere R1
2. P2 adquiere R2

3. P3 adquiere R3
4. P1 requiere R2
5. P2 requiere R3
6. P3 requiere R1

¿Existe deadlock si P1 libera R1 antes del paso 4? Justificar en base al grafo resultante y mostrar el progreso (si existiera) de los procesos hasta su finalización.

Ejercicio 14: Implementar el productor-consumidor usando procesos y colas de mensajes POSIX. Ver https://man7.org/linux/man-pages/man7/mq_overview.7.html

Un productor debe bloquearse cuando el buffer (cola) está lleno. Similarmente, un consumidor debe bloquearse cuando el buffer (cola) está vacío. Usar las señales SIGSTOP y SIGCONT para suspender/continuar los procesos.

Definir el formato de mensajes a conveniencia.

Ayuda: Un proceso puede auto-bloquearse enviando la señal SIGSTOP a sí mismo.